UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA Y DISEÑO INDUSTRIAL

Grado en Ingeniería Electrónica Industrial y Automática

PROYECTO DE INFORMÁTICA INDUSTRIAL

DESARROLLO DEL VIDEOJUEGO ARCHON MEDIANTE LA PROGRAMACIÓN ORIENTADA A OBJETOS EN C++

RANCHO

Julián Osiniri Juárez

Álvaro Pérez Ruano

Miguel Vázquez Aguado

Jaime Ladero Blanco

David San Román González

Madrid, Mayo de 2026

ÍNDICE

1. INTRODUCCIÓN 2

1.1. Origen de "Rancho" 2

1.2. Objetivos del proyecto 2

2. MANUAL DE INSTRUCCIONES 2

2.1. Requisitos del sistema y dependencias 2

2.2. Guía de inicio 2

2.3. Controles 2

2.4. Reglas del juego 2

3. ARQUITECTURA DEL SOFTWARE (POO) 2

3.1. Desacople entre lógica y motor gráfico 2

3.2. Patrones de diseño 3

3.3. Estructura de clases 3

4. LÓGICA DE JUEGO 3

4.1. Fase estratégica (Tablero) 3

4.2. Fase de acción (Arena) 3

4.3. Las piezas (Animales) 3

5. METODOLOGÍA Y CONTROL DE VERSIONES 4

5.1. Flujo de trabajo (Gitflow) 4

5.2. Integración y revisión de código 4

5.3. Estructura del proyecto 4

6. ELEMENTOS ORIGINALES Y OPCIONALES 4

6.1. Personalización temática 4

6.2. Funcionalidades extra 4

7. CONCLUSIONES 4

1. INTRODUCCIÓN

1.1. Origen de "Rancho"

Presentar la temática de animales de rancho como variante original de Archon.

1.2. Objetivos del proyecto

Cumplir los requisitos mínimos y hacer algunos opcionales.

2. MANUAL DE INSTRUCCIONES

2.1. Requisitos del sistema y dependencias

Instrucciones detalladas para configurar Visual Studio 2026, freeglut y ETSIDIlib (también cómo enlazar dependencias no caben en el .zip).

2.2. Guía de inicio

Pasos para compilar el proyecto.

2.3. Controles

División de inputs (WASD y Flechas), uso del menú/HUD.

2.4. Reglas del juego

3. ARQUITECTURA DEL SOFTWARE (POO)

3.1. Desacople entre lógica y motor gráfico

Explicación de cómo la lógica trabaja de forma independiente a freeglut/ETSIDIlib, que está solo en la clase Renderizador.

3.2. Patrones de diseño

Máquina de Estados y Patrón Comando en la clase coordinadora, Juego.

Gestión de memoria y recursos (RAII). Patrones GRASP.
Regla de los Tres/Cinco/Cero (falta decidir cual, creo que Cero es la mejor).
Principios SOLID.

3.3. Estructura de clases

Abstracción, Encapsulamiento, Herencia y Polimorfismo.

El coordinador (Juego) y la gestión de fases (Menú, Tablero, Batalla).

Jerarquía polimórfica: clase abstracta Animal y sus hijas (Gallina, Cerdo, etc.), usando métodos virtuales puros y enlace dinámico.

Estructuras de control: Tablero (matriz de punteros) y Arena (fase de acción).

4. LÓGICA DE JUEGO

4.1. Fase estratégica (Tablero)

Validación de movimientos, oscilación de casillas y gestión de turnos.

4.2. Fase de acción (Arena)

Algoritmos de colisión (AABB), confinamiento en los límites de la pantalla y gestión dinámica de proyectiles/ataques.

4.3. Las piezas (Animales)

Características de cada animal, tipo, vida… Foto del animal.

Para el granjero, habilidades equivalentes a los hechizos del Archon original.

5. METODOLOGÍA Y CONTROL DE VERSIONES

5.1. Flujo de trabajo (Gitflow)

Uso de ramas main, develop y feature/.

5.2. Integración y revisión de código

Explicación del sistema de Pull Requests

5.3. Estructura del proyecto

Distribución profesional de directorios (/include, /src, /assets, etc.).

6. ELEMENTOS ORIGINALES Y OPCIONALES

6.1. Personalización temática

Elementos únicos de "Rancho" (gráficos, música, sonidos).

6.2. Funcionalidades extra

Menú principal, créditos…

7. CONCLUSIONES

Retos superados.

Trabajo futuro, posibles mejoras…